

Ilmarinen: A Symbolic Technical Reference

Contracts, Schemas, Primitives, and Selection Operators

Technical reference notes

July 22, 2026

Abstract

This is a reference catalogue of what the `ilmarinen` package computes, expressed symbolically. It enumerates the eight computational *contracts* (contracts), the `AllGraph` controller and its selection operators, every *schema* and its meta-cell algebra, and every *primitive* with the exact tensor operation it realizes. Notation follows the analytical report (*the action* $J = R + \mu\Omega$; the mixture weight α as a point on the primitive simplex) and the implementation report (module and symbol names in `typewriter`). The goal is a compact map from code element to mathematical operation, not a derivation; derivations live in the other two reports and are cited by section name where relevant.

Contents

1	Common objects and the schema algebra	1
2	Contracts and the <code>AllGraph</code> controller	2
3	Sequence contract	3
4	Grid contracts: spatial, volumetric, 4d	4
5	Graph contract	4
6	Equivariant contract	5
7	Set contract	6
8	Operator contract	6
9	Selection operators (instances of $J = R + \mu\Omega$)	7
10	Deriving α , co-adaptation, and discretization	8
11	Read-outs and heads	8

1 Common objects and the schema algebra

Data container. A dataset is wrapped in `AllData`, which fixes the *type* by how it is built: `dense_tensor(X)` for a fixed-size array X (a grid of some rank), `graphs({Vi}, {Ei})` for node features with

edges, `point_sets`($\{V_i\}$) for unordered sets, and `functions`(a, y) for field-valued input/target pairs sampled on a grid (the neural-operator interface). Positions $\{\mathbf{x}_i\} \subset \mathbb{R}^3$ may accompany graphs/sets to signal geometry.

Primitive. A primitive is a map $f_p : \mathcal{X} \rightarrow \mathcal{X}$ on a contract’s carrier space, type-preserving so that primitives compose across depth and mix within a layer. Each contract c has a finite *vocabulary* $\mathcal{P}_c = \{f_p\}$; the vocabularies are catalogued in §3–§8.

Meta-cell (the relaxation). A cell places the vocabulary in parallel and mixes it by a softmax over a learnable logit vector $\boldsymbol{\alpha} \in \mathbb{R}^{|\mathcal{P}_c|}$. Writing $w = \text{softmax}(\boldsymbol{\alpha})$, the cell map is

$$\Phi_{\boldsymbol{\alpha}}(h) = \sum_{p \in \mathcal{P}_c} w_p f_p(h), \quad w_p = \frac{e^{\alpha_p}}{\sum_q e^{\alpha_q}}. \quad (1)$$

Every contract reuses (1) verbatim; the cell classes are `UnifiedCell`, `_GraphCell`, `_EquivCell`, `UnifiedSetCell`, `_OperatorCell`, and so on, differing only in the carrier space and $\mathcal{P}_c$. The mixture is a convex combination on the simplex $\Delta^{|\mathcal{P}_c|-1}$, so at a vertex ($w \rightarrow e_p$) the cell collapses to a single primitive.

Schema (depth composition). A depth- L schema stacks L independent cells; layer ℓ consumes the previous layer’s mixed output,

$$h^{(0)} = \text{input}, \quad h^{(\ell)} = \Phi_{\boldsymbol{\alpha}^{(\ell)}}(h^{(\ell-1)}), \quad \ell = 1, \dots, L, \quad (2)$$

each layer carrying its own $\boldsymbol{\alpha}^{(\ell)}$ (independent per-layer selection). A read-out head g maps the top representation to the prediction $\hat{y} = g(h^{(L)})$ (§11). The first layer maps input width C_{in} to hidden width K ; subsequent layers are $K \rightarrow K$.

The action. All selection in `AllGraph` minimizes a single description-length action

$$J = R + \mu \Omega, \quad (3)$$

with R the empirical risk (cross-entropy or MSE) and Ω a complexity charge; the different selection stages (§9) are instances of (3) with different Ω (structure, width, depth, mixture sparsity, spectral modes). The price μ is the accuracy/complexity exchange rate (*Pareto slope*); $\mu = 0$ recovers pure fit.

2 Contracts and the AllGraph controller

The eight contracts. `AllGraph._ARENAS` enumerates the contracts, each an equivariance/interface class with its own carrier space, symmetry group G , and vocabulary:

contract	carrier	symmetry G	§
<code>sequence</code>	(b, T, C_{in}) time series	time translation (soft)	3
<code>spatial</code>	(b, C_{in}, H, W) image	2D translation	4
<code>volumetric</code>	$(b, C_{\text{in}}, D, H, W)$ volume	3D translation	4
<code>4d</code>	$(b, C_{\text{in}}, T, D, H, W)$ space–time	4D translation	4
<code>graph</code>	nodes (N, F) + edges E	node permutation S_n	5
<code>equivariant</code>	geometric graph $(s, \mathbf{v}, \mathbf{x}, E)$	$S_n \times \text{SO}(3)$	6
<code>set</code>	elements (N, F) + membership	element permutation S_n	7
<code>operator</code>	fields $a(\mathbf{x}) \mapsto u(\mathbf{x})$	discretization-invariance	8

Routing ρ . `route( $\mathcal{D}$ )` maps data to a contract by its kinematics: rank and structure of the tensor decide the grid family (`sequence/spatial/volumetric/4d`), presence of edges lifts to `graph`, positions lift to `equivariant`, edgeless node collections route to `set`, and field pairs to `operator`. A flat vector (b, d) routes to `sequence` (unstructured rank-1). If the routed contract is disabled, `_resolve_enabled_fallback` degrades along a constructibility order (e.g. `equivariant` $\rightarrow$ `graph` $\rightarrow$ `set` $\rightarrow$ `sequence`), dropping the structure the fallback cannot honor.

Tie-break τ . When several contracts are admissible (e.g. a geometric graph could be run as `set`, `graph`, or `equivariant`), `tiebreak` trains each admissible contract solo and selects by the priced action

$$c^* = \arg \min_c \left[R(c) + \mu_c \Omega_{\text{struct}}(c) \right], \quad (4)$$

i.e. the contract whose structural prior pays for itself in fit (`machinery/contract_md1.py`; Bayesian-evidence reading in `contract_evidence.py`).

Controller API. `AllGraph` exposes: `route` and `tiebreak` (contract choice), `select_architecture` and `select_architecture_by_area` (size choice, §9), `fit` (build the chosen schema, train, optionally price the mixture and discretize), `apply_canonicalization` (pose alignment for geometric data), and `explain` (attribution/report). The controller holds width K , depth L , seed, and the enabled-contract restriction.

3 Sequence contract

Carrier $(b, T, C_{\text{in}}) \rightarrow (b, T, K)$; module `schema.py`. Recurrences carry a state $h_t \in \mathbb{R}^K$; nonrecurrent primitives act on the whole sequence. Weights are initialized with an edge-of-chaos gain $\sigma_w^2 \approx 1.76$ (the critical variance of the analytical report). Table 1 lists the eleven cores; $x_t \in \mathbb{R}_{\text{in}}^C$ is the input at time t , and all outputs are in $\mathbb{R}^K$.

Table 1: Sequence primitives $\mathcal{P}_{\text{sequence}}$ (`_SEQ_CORES`).

name	class	operation
<code>plain</code>	<code>_PlainSeq</code>	$h_t = \tanh(W_x x_t + W_h h_{t-1})$ (vanilla RNN)
<code>gated</code>	<code>_GatedSeq</code>	GRU: $r, z = \sigma(\cdot)$, $n = \tanh(W_x^n x_t + r \odot W_h^n h_{t-1})$, $h_t = (1-z) \odot n + z \odot h_{t-1}$
<code>lstm</code>	<code>_LSTMSeq</code>	LSTM gates $i, f, o = \sigma(\cdot)$, $c_t = f \odot c_{t-1} + i \odot \tanh g$, $h_t = o \odot \tanh(c_t)$; optional chrono-init of b_f [24]
<code>conv</code>	<code>_ConvSeq</code>	causal 1D conv: $h = \tanh(\text{Conv1d}_k(\text{pad } x))$, kernel $k=5$
<code>dilconv</code>	<code>_DilatedConvSeq</code>	multiscale causal conv $\tanh(\big\ _d \text{Conv1d}_{k,d}(x)\big\)$, $d \in \{1, 2, 4, 8\}$, then 1×1 mix (TCN [1], InceptionTime [7])
<code>attention</code>	<code>_AttentionSeq</code>	causal self-attention $A = \text{softmax}(\text{tril}(QK^\top/\sqrt{d}))$, $h = \tanh(W_o AV)$
<code>dense</code>	<code>_DenseSeq</code>	per-step affine $h_t = \tanh(W x_t + b)$ (no time mixing)
<code>norm</code>	<code>_NormSeq</code>	$h_t = \tanh(W \text{LN}(x_t))$ (per-step LayerNorm stabilizer)
<code>linssm</code>	<code>_LinearSSMSeq</code>	diagonal LTI SSM $h_t = a \odot h_{t-1} + B x_t$, $y_t = \tanh(C h_t)$, $a = \sigma(\cdot) \in (0, 1)$ [11, 23]; scan $h_t = a^t \sum_{s \leq t} a^{-s} B x_s$

name	class	operation
selssm	<code>_</code> <code>SelectiveSSMSeq</code>	Mamba/S6, input-dependent decay $\bar{A}_t = e^{-\Delta_t A}$; state $h_t = \bar{A}_t \odot h_{t-1} + (\Delta_t B_t) \odot x'_t$, gated readout $y_t = \tanh(C_t \odot h_t + D \odot x'_t)$ [10]
spectral	<code>_SpectralSeq</code>	rFFT _t log-power $\rightarrow$ (mean max) over freq $\rightarrow \tanh(W \cdot)$ broadcast over t (frequency-diagonal)

The mixture (1) contracts the P per-primitive sequence outputs, $\Phi_\alpha(x) = \sum_p w_p f_p(x)$ (an `einsum p, pbtw  $\rightarrow$  btw`). When every selected core is recurrent, the stack streams one timestep at a time (`step`); a nonrecurrent core (`conv/dilconv/attention/dense/norm/spectral`) forces whole-sequence evaluation.

4 Grid contracts: spatial, volumetric, 4d

Carriers (b, C_{in}, H, W) , $(b, C_{\text{in}}, D, H, W)$, $(b, C_{\text{in}}, T, D, H, W)$; modules `unified_spatial_/volumetric_/grid4d_schema.py`. The three vocabularies are the same primitive *ideas* at ranks $r = 2, 3, 4$: local convolution (weight sharing under translation), separable/depthwise variants, pointwise channel mixing, dense (no locality), normalization, and attention. All convolutional cores apply tanh after the (padded) convolution.

name	operation (Conv _r = rank- r convolution)
conv2d/3d	$\tanh(\text{Conv}_r^k(x))$, $k=3$ local kernel (weight-shared under rD translation)
conv2d_k5, k7	same with $k=5, 7$ (larger receptive field; distinct simplex points)
conv3d_k5	rank-3, $k=5$
conv_dw	depthwise conv (per-channel spatial) then $1 \times \dots$ mix (separable)
atrous	dilated conv, rates $\{1, 2, 4\}$ (multiscale 2D receptive field)
pointwise	1^r conv = per-voxel channel mixing $\tanh(Wx)$
conv4d, _kt1	exact 4D conv (space-time coupling); <code>_kt1</code> sets $K_t=1$ (space-time separable)
dense	affine over the flattened grid (no locality; $O((C_{\text{in}} \cdot \text{vol})^2)$)
norm	normalization + affine (stabilizer)
attention	flattened-grid self-attention (content routing over positions)

`conv4d` is realized exactly by decomposing the 4D correlation into a temporal loop of 3D convolutions (there is no native `conv4d`); the space-time separable `conv4d_kt1` drops temporal mixing. Depth composition and the α -mixture are (2)–(1) as elsewhere.

5 Graph contract

Carrier: node features $X \in \mathbb{R}^{N \times F}$, directed edges E with rows `[src, dst]` (messages flow `src $\rightarrow$ dst`); module `graph_schema.py`. Let $\mathcal{N}(v) = \{u : (u, v) \in E\}$ and let $\bigoplus$ denote a permutation-invariant neighbor aggregation. All cores map $\mathbb{R}^{N \times F} \rightarrow \mathbb{R}^{N \times K}$ and are S_n -equivariant (a node permutation permutes the rows identically).

name	class	operation (per node v , $h = Wx$)
gcn	<code>_GCNGraph</code>	isotropic mean: $h_v + \text{mean}_{u \in \mathcal{N}(v)} h_u$ [14]
sage	<code>_SAGEGraph</code>	max-pool: $W_s x_v + \max_{u \in \mathcal{N}(v)} \text{ReLU}(W_m x_u)$ [12]
gin	<code>_GINGraph</code>	sum + MLP: $\text{MLP}((1+\epsilon)x_v + \sum_{u \in \mathcal{N}(v)} x_u)$ [29]
pna	<code>_PNAGraph</code>	multi-aggregator $\{\text{mean}, \text{max}, \text{min}, \text{std}\} \times$ degree-scalers $\{1, \log(d+1), 1/\log(d+1)\}$, then $h_v + W_p(\cdot)$ [5]
gat	<code>_GATGraph</code>	attention: $a_{uv} = \text{softmax}_u(\text{LeakyReLU}(\mathbf{a}^\top [h_u \ h_v]))$, $h_v + \sum_u a_{uv} h_u$ [27]
dense	<code>_DenseGraph</code>	per-node Wx_v (edges ignored; no-structure baseline)
norm	<code>_NormGraph</code>	$\text{LN}(Wx_v)$ (over-smoothing stabilizer)

Graphs are batched by concatenating nodes and offsetting edge indices; a per-graph readout (§11) pools nodes to a graph vector. The aggregators differ in their 1-WL discriminative power: **gin** (injective sum) $\succeq$ **gcn** (mean), and **pna**’s multi-aggregator design lifts the single-aggregator injectivity gap.

6 Equivariant contract

Carrier: geometric graph with type-0 (scalar) features $s \in \mathbb{R}^{N \times C_0}$ and type-1 (vector) features $\mathbf{v} \in \mathbb{R}^{N \times C_1 \times 3}$, positions $\mathbf{x} \in \mathbb{R}^{N \times 3}$, edges E ; module `equivariant_graph_schema.py`. All cores are $\text{SO}(3)$ -equivariant and type-preserving: under $\mathbf{x} \mapsto \mathbf{R}\mathbf{x}$, scalars are invariant and vectors rotate, $\mathbf{v} \mapsto \mathbf{v}\mathbf{R}^\top$. Edge geometry enters through the unit direction $\hat{\mathbf{r}}_{uv} = (\mathbf{x}_v - \mathbf{x}_u)/\|\mathbf{x}_v - \mathbf{x}_u\|$ and length d_{uv} .

Equivariant building blocks. Type-preserving linear `_EquivLinear`: $s \mapsto sW_s + b_s$, $\mathbf{v} \mapsto \mathbf{v}W_v$ (channel mix shared across the $2\ell+1$ components, no vector bias). Gated nonlinearity `_GatedNonlin`: $s \mapsto \tanh s$, $\mathbf{v} \mapsto \sigma(\text{Lin}(s)) \odot \mathbf{v}$ (an *invariant* gate scales vectors, preserving direction). Norm `_EquivNorm`: $\text{LN}(s)$ on scalars; $\mathbf{v}$ divided by the RMS of its per-channel invariant norms $\|\mathbf{v}_c\|$.

Clebsch–Gordan message. `_tensor_product_message` couples source features with the edge spherical harmonics $Y_0=1$, $Y_1=\hat{\mathbf{r}}$ for $\ell \leq 1$:

$$\underbrace{s\text{-out}}_0: \{s_u, \langle \mathbf{v}_u, \hat{\mathbf{r}} \rangle\}, \quad \underbrace{\mathbf{v}\text{-out}}_1: \{s_u \hat{\mathbf{r}}, \mathbf{v}_u \times \hat{\mathbf{r}}\}, \quad (5)$$

i.e. the CG paths [25, 9] $0 \times 0 \rightarrow 0$, $1 \times 1 \rightarrow 0$ (dot), $0 \times 1 \rightarrow 1$ (scale), $1 \times 1 \rightarrow 1$ (cross). Table 2 lists the eight cores.

Table 2: Equivariant primitives $\mathcal{P}_{\text{equivariant}}$ (`_EQUIV_CORES`).

name	class	operation
e_tp	<code>_TPMessage</code>	CG message (5) weighted by Gaussian-RBF radial $w(d)$, summed over $\mathcal{N}(v)$, then <code>_EquivLinear</code> mix (NequIP [4], MACE [3])

name	class	operation
<code>e_kan</code>	<code>_KANMessage</code>	as <code>e_tp</code> but $w(d)$ is a <i>learnable</i> piecewise-linear spline $\text{lerp}(\text{knots}, d)$ (equivariant KAN [13, 19]; radial curve inspectable)
<code>e_painn</code>	<code>_PaiNNMessage</code>	scalar MLP $\phi(s_u)$ split into (s-msg, a, b), radial-gated; $v\text{-msg} = a \hat{r} + b v_u$, summed, mixed (PaiNN [22])
<code>e_painn_kan</code>	<code>_PaiNNKAN</code>	PaiNN message with a learnable-spline radial gate (KAN sibling of <code>e_painn</code>)
<code>e_mean</code>	<code>_EquivMean</code>	CG message with degree-normalized <i>mean</i> aggregation, no RBF (isotropic equivariant “gcn”)
<code>e_linear</code>	<code>_EquivSelf</code>	<code>_EquivLinear</code> self-interaction, no message passing (equivariant “dense”)
<code>e_gate</code>	<code>_EquivGate</code>	<code>_EquivLinear</code> then <code>_GatedNonlin</code> (equivariant nonlinear update)
<code>e_norm</code>	<code>_EquivNormCore</code>	<code>_EquivLinear</code> then <code>_EquivNorm</code> (stabilizer)

The `_l2` variant (`equivariant_graph_schema_l2.py`) extends the type ladder to $\ell \leq 2$ (adds rank-2 tensor features C_2) [8, 25]. Scalable non-CG realizations (vector-neuron [6] / G-RepsNet [2] style) live in `models/scalable_equivariant.py`, and priced *approximate* equivariance (a free non-equivariant pathway gated by a breaking-power price) in `models/approximate_equivariance.py`.

7 Set contract

Carrier: elements $X \in \mathbb{R}^{N \times F}$ with a batch map $b : \{1:N\} \rightarrow \{1:n_{\text{sets}}\}$; module `set_schema.py`. Cores are S_n -equivariant element maps $\mathbb{R}^{N \times F} \rightarrow \mathbb{R}^{N \times K}$; a permutation-invariant pool at readout gives the per-set output (Deep Sets [30] factorization $\rho(\bigoplus_i \phi(x_i))$).

name	operation ($h = Wx$, pooling within each set)
<code>deepsets</code>	element MLP $\phi(h)$; invariant pool applied at readout ($\rho(\bigoplus_i \phi(x_i))$)
<code>element_mlp</code>	per-element MLP $\phi(h)$ (minimal set map / stabilizer)
<code>sab</code>	Set-Attention Block [16]: $a = \text{LN}(h + \text{MHA}(h))$, $\text{LN}(a + \text{FF}(a))$
<code>isab</code>	Induced SAB: m learned inducing points attend to elements, elements attend back ($O(Nm)$)
<code>pma_block</code>	element MLP feeding a Pooling-by-Multihead-Attention readout
<code>norm</code>	element-wise normalization (parameter-light stabilizer)

Multihead attention `_MHA` masks cross-set pairs, scores $q_k = \langle q, k \rangle / \sqrt{d_k}$ with a $-\infty$ mask off-set, so equivariance holds by construction; the pool $\bigoplus \in \{\text{sum}, \text{mean}, \text{max}, \text{FSPool}\}$ selects the invariant readout, where FSPool [31] sorts each feature across elements and reads an order-weighted descriptor (retains distributional shape, not just a moment).

8 Operator contract

Carrier: input field $a(\mathbf{x})$ and target field $u(\mathbf{x})$ sampled on a grid, lifted to a channel field $h \in \mathbb{R}^{b \times K \times \text{grid}}$; module `operator_schema.py`. Cores are resolution-independent maps $h \mapsto h$ (a trained

operator transfers zero-shot across grid resolutions). Let $\mathcal{F}$ be the (real) FFT along the spatial axes.

name	class	operation
fourier	<code>_FourierOp</code>	FNO [17] layer: $\text{ReLU}(\mathcal{F}^{-1}(R_{\leq m} \mathcal{F}h) + W_{1 \times 1}h)$; keep m low modes with learnable complex weights R
fourier_wide	<code>_FourierWideOp</code>	FNO layer with more modes $m' > m$ (higher spectral capacity)
local	<code>_LocalOp</code>	pointwise (Nemytskii) operator $\text{ReLU}(W_{1 \times 1}h)$ (no global coupling)
deeponet	<code>_DeepONetOp</code>	branch $b_k = \text{Br}(\text{mean}_{\text{grid}} h)$, trunk $t_k(\mathbf{y}) = \text{Tr}(\mathbf{y})$; $u(\mathbf{y}) = \sum_k b_k t_k(\mathbf{y})$ (DeepONet [20]; mesh-free)

The spectral cores exist at ranks 1/2/3 (`_SpectralConv1d/2d/3d`); **deeponet**'s trunk is a continuous function of the query coordinate, so it evaluates off the training grid (mesh-free). The per-primitive mode budget m is itself a priced degree of freedom (§9, spectral selection).

9 Selection operators (instances of $J = R + \mu\Omega$)

The controller turns the fixed hyperparameters (contract, width, depth, mixture support, mode count) into *outputs* of priced measurements. Each is a marginal-value rule on (3).

Contract. (4): argmin over admissible contracts of $R(c) + \mu_c \Omega_{\text{struct}}(c)$; Ω_{struct} is the description length of the contract's structural prior (`contract_md1.py`), with a Bayesian-evidence equivalent (`contract_evidence.py`). Optionally (`price_singular`, `singular_md1.py`) the charge is augmented by the *functional* code length $\omega_{\text{func}} = \lambda \log n$ from singular learning theory [15, 28], $\Omega_{\text{total}}(c) = \Omega_{\text{struct}}(c) + (\mu_s/\mu_c)\lambda_c \log n$, where λ_c is the local learning coefficient (RLCT) of the contract's converged candidate; a non-negativity clamp and a non-convergence guard keep a noisy $\hat{\lambda}$ from entering J . This prices the effective, not the nominal, dimensionality of the fitted function, and is second-order to Ω_{struct} on realistic converged data.

Width K^* . `select_architecture` sweeps width and reads the marginal value of neurons; the analytical certificate sets the price scale. `width_sparsity.py` realizes a two-layer fit $f(x) = \sum_k a_k \tanh(w_k^\top x + b_k)$ by conditional-gradient atom insertion, adding the neuron of maximal residual correlation while the dual certificate $\eta = A^*q/\lambda$, $\|\eta\|_\infty \leq 1$, is unsaturated (`dual_certificate.py`, `tv_certificate.py`). A boundary-extension guard widens the grid when the optimum sits on its edge.

Depth L^* . `machinery/priced_depth.py`: measure the depth curve $S(L)$ (validation loss) over seeds and add layers while the per-layer marginal exceeds the price,

$$L^* = \max \left\{ L : -(S(L) - S(L-1)) > \mu \right\}, \quad (6)$$

or, price-free, the last depth whose marginal clears n_{se} standard errors (`significant_elbow`, $n_{\text{se}}=2$ the empirically safe bar). `select_architecture_by_area` minimizes total area (width $\times$ depth) subject to a within-noise accuracy tolerance and extends depth past the grid boundary up to a cap when the deeper column beats the noise floor.

Mixture sparsity. `sparsity_priced_alpha.py`: charge the in-layer mixture its description length so a layer keeps extra primitives only when they pay,

$$J(\alpha) = R(\alpha) + \mu \Omega(\alpha), \quad \Omega(\alpha) = H(w) \text{ or } \|w\|_0^{\text{soft}}, \quad w = \text{softmax}(\alpha), \quad (7)$$

driving w toward a simplex vertex (a single primitive) unless the fit rewards a genuine mixture. This is the deployment analogue of the width and depth charges: one more Ω -term in (3).

Spectral modes. `spectral_selection.py`: for the operator contract, price the number of retained Fourier modes m by the same marginal-value rule, so spectral capacity is selected, not fixed.

Price μ . `price_selection.py`: μ is the exchange rate in the scalarized multi-objective ($\min R, \min \Omega$); sweeping μ traces the Pareto frontier, and an “elbow” / L-curve criterion picks the operating point. $\mu=0$ is pure fit.

10 Deriving α , co-adaptation, and discretization

α as a Gibbs measure. `gibbs_alpha.py` identifies the softmax mixture weight as the stationary measure of a free-energy flow on the simplex: with free energy $\mathcal{F}_\alpha = \langle \alpha, \Psi \rangle - \frac{1}{\beta} H(\alpha)$, the minimizer is the Gibbs form $w_p \propto e^{-\beta \Psi_p}$, its Fisher–Rao gradient flow is the replicator equation, and DARTS [18] is that flow in a mildly preconditioned metric. The variational principle forces the per-primitive energy Ψ_p to be the *solo* marginal risk.

Co-adaptation. `coadapt.py`: in a supernet all primitives run and mix, so their gradients couple and a high-capacity primitive is over-credited. The fix reads the *solo* marginal (each primitive scored alone) as the selection signal and arbitrates a clean single-primitive winner, aligning the selection energy with the α -derivation above.

Bilevel selection. `bilevel.py`: weights are trained on a w -split and α on a disjoint α -split (three-way split with a held-out test), removing the training-loss bias that selects capacity; `discretize_and_finetune` then collapses the mixture to its argmax primitive and fine-tunes, converting a soft selection into a deployed discrete architecture. `robust_discretize.py` additionally collapses a redundant depth (adjacent layers selecting the same primitive) when doing so does not raise the loss beyond noise.

Singular complexity. `singular_complexity.py`: the classical $\Omega = \frac{k}{2} \log n$ (parameter count) is replaced, where a local minimum is genuinely reached, by the local learning coefficient (RLCT) λ of singular learning theory [28, 15], $\Omega = \lambda \log n$; a negative measured λ signals non-convergence rather than low complexity, so the estimate is only used at a converged minimum and only comparatively across independently-converged architectures.

11 Read-outs and heads

The top representation $h^{(L)}$ is mapped to a prediction by a contract-appropriate head g . *Sequence*: **last** $h_T^{(L)}$ (recall/copy), **mean** $\frac{1}{T} \sum_t h_t^{(L)}$ (classification), or **flatten** $\text{vec}(h^{(L)})$ (position-aware), then a linear map to n_{out} ; optional deep supervision attaches an early linear head to each

layer. *Graph/set*: a permutation-invariant pool over nodes/elements $\oplus \in \{\text{mean}, \text{sum}, \text{max}\}$ to a graph/set vector, then linear. *Equivariant*: an *invariant* read-out of the scalar channels (and vector norms) so the target is $\text{SO}(3)$ -invariant (energies) or, for vector targets, a type-1 output. *Grid*: global pool over the spatial axes then linear. *Operator*: the field itself is the output (a 1×1 projection to the target channels on the grid). A symbolic read-out option (`core/symbolic_readout.py`) fits a closed-form expression to the pooled features when an interpretable law is sought.

Auxiliary pipelines. Beyond the eight forward contracts, `core/` provides the symmetry-discovery pipeline (linear/ $\text{SO}(3)$ and nonlinear-latent breaking tests, canonicalization, metric/group discovery), the interpretability stack (feature attribution, the inspectable KAN radial curves of §6), and the information-theoretic diagnostics (IB [26]/RG-flow [21], correlation length, mean-field and exponent estimates) used to instrument the selections above. A family of singular/thermodynamic read-outs sits on the same objects (`machinery/`): the local learning coefficient λ at the optimum and its training trajectory $\lambda(t)$; the single free-energy form $F = \langle \rho, E \rangle - \beta^{-1} H(\rho)$ with its three decoupled temperatures ($\beta_W = 1/\log n$ at the weights, β_A at the primitive readout, $\beta_C = 1$ at the contracts) and a consistency guard; the response susceptibility (readout specific heat and the first-order contract transitions in the price); and the effective-dimension ledger, one participation-ratio functional $1/\sum p^2$ read across the coarse-graining axis (data modes, primitive mixture) with λ as the model-level cap. All are pure read-outs that change no selection; they are catalogued in the implementation report.

Provenance and references

`ilmarinen` is an integrative repackaging: each primitive and selection rule realizes an idea from the published literature, recombined under the common action $J = R + \mu\Omega$ and the meta-cell relaxation (1). The citations attached to the primitive tables and selection operators above point to the originating works; the consolidated list follows. The package’s own contribution is the unification and the honest empirical auditing of when each component pays for itself, not the primitives themselves.

References

- [1] S. Bai, J. Z. Kolter, and V. Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling, 2018. TCN.
- [2] S. Basu, S. Lohit, and M. Brand. G-repsnet: A lightweight construction of equivariant networks for arbitrary matrix groups. In *TMLR*, ; , 2024, 2025.
- [3] I. Batatia, D. P. Kovács, G. Simm, C. Ortner, and G. Csányi. Mace: Higher order equivariant message passing neural networks for fast and accurate force fields. In *NeurIPS*, 2022.
- [4] S. Batzner et al. E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature Communications* 13, 2453, 2022. NequIP.
- [5] G. Corso, L. Cavalleri, D. Beaini, P. Liò, and P. Veličković. Principal neighbourhood aggregation for graph nets. In *NeurIPS*, 2020. PNA.
- [6] C. Deng et al. Vector neurons: A general framework for $\text{so}(3)$ -equivariant networks. In *ICCV*, 2021.

- [7] H. Ismail Fawaz et al. Inceptiontime: Finding alexnet for time series classification. *Data Mining and Knowledge Discovery*, 34, 2020.
- [8] M. Finzi, M. Welling, and A. G. Wilson. A practical method for constructing equivariant multilayer perceptrons for arbitrary matrix groups. In *ICML*, 2021. EMLP.
- [9] M. Geiger and T. Smidt. e3nn: Euclidean neural networks, 2022.
- [10] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023. S6/selective SSM.
- [11] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. In *ICLR*, 2022. S4.
- [12] W. L. Hamilton, R. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *NeurIPS*, 2017. GraphSAGE.
- [13] L. Hu, Y. Wang, and Z. Lin. Incorporating arbitrary matrix group equivariance into kans. In *(EKAN). ICML*, 2025. EKAN.
- [14] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017. GCN.
- [15] E. Lau, D. Murfet, and S. Wei. The local learning coefficient: A singularity-aware complexity measure, 2023.
- [16] J. Lee et al. Set transformer: A framework for attention-based permutation-invariant neural networks. In *ICML*, 2019. SAB/ISAB/PMA.
- [17] Z. Li et al. Fourier neural operator for parametric partial differential equations. In *ICLR*, 2021. FNO.
- [18] H. Liu, K. Simonyan, and Y. Yang. Darts: Differentiable architecture search. In *ICLR*, 2019.
- [19] Z. Liu et al. Kan: Kolmogorov–arnold networks, 2024.
- [20] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via deepnet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3, 2021.
- [21] P. Mehta and D. J. Schwab. An exact mapping between the variational renormalization group and deep learning, 2014.
- [22] K. T. Schütt, O. T. Unke, and M. Gastegger. Equivariant message passing for the prediction of tensorial properties and molecular spectra. In *ICML*, 2021. PaiNN.
- [23] J. T. H. Smith, A. Warrington, and S. W. Linderman. Simplified state space layers for sequence modeling. In *ICLR*, 2023. S5.
- [24] C. Tallic and Y. Ollivier. Can recurrent neural networks warp time? In *ICLR*, 2018. chrono-init.
- [25] N. Thomas et al. Tensor field networks: Rotation- and translation-equivariant neural networks for 3d point clouds, 2018.

- [26] N. Tishby and N. Zaslavsky. Deep learning and the information bottleneck principle. *IEEE ITW*, 2015.
- [27] P. Veličković et al. Graph attention networks. In *ICLR*, 2018. GAT.
- [28] S. Watanabe. *Algebraic Geometry and Statistical Learning Theory*. Cambridge University Press, 2009. singular learning / RLCT.
- [29] K. Xu, W. Hu, J. Leskovec, and S. Jegelka. How powerful are graph neural networks? In *ICLR*, 2019. GIN.
- [30] M. Zaheer et al. Deep sets. In *NeurIPS*, 2017.
- [31] Y. Zhang, J. Hare, and A. Prügel-Bennett. Fspool: Learning set representations with feature-wise sort pooling. In *ICLR*, 2020.